



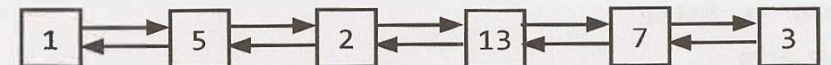
▶ Linked List

By Jorge Galvis

Linked List

- It represents a sequence of nodes
- Nodes may not be next to each other
- It does not provide constant time to access a particular node

```
class Node {  
    Node next = null;  
    int data;  
  
    public Node(int d) {  
        data = d;  
    }  
  
    void appendToTail(int d) {  
        Node end = new Node(d);  
        Node n = this;  
        while (n.next != null) {  
            n = n.next;  
        }  
        n.next = end;  
    }  
}
```



Deleting a node

- Check null pointer
- Update the head / tails as needed
- Free memory

```
Node deleteNode(Node head, int d) {  
    Node n = head;  
  
    if (n.data == d) {  
        return head.next; /*moved head */  
    }  
  
    while (n.next != null) {  
        if (n.next.data == d) {  
            n.next = n.next.next;  
            return head; /*head didn't change */  
        }  
        n = n.next;  
    }  
    return head;  
}
```

The runner technique

- The "runner" (or second pointer) technique is used in many linked list problems. The runner technique means that you iterate through the linked list with two pointers simultaneously, with one ahead of the other. The "fast" node might be ahead by a fixed amount, or it might be hopping multiple nodes for each one node that the "slow" node iterates through.

For example, suppose you had a linked list $a_1 \rightarrow a_2 \rightarrow \dots \rightarrow a_n \rightarrow b_1 \rightarrow b_2 \rightarrow \dots \rightarrow b_n$ and you wanted to rearrange it into $a_1 \rightarrow b_1 \rightarrow a_2 \rightarrow b_2 \rightarrow \dots \rightarrow a_n \rightarrow b_n$. You do not know the length of the linked list (but you do know that the length is an even number).

Operations

- Operations at the head and at the tails are faster

Cheat Sheet



	Big O Complexity
lookup	$O(n)$
append (w/ tail)	$O(1)$
append (w/o tail)	$O(n)$
prepend	$O(1)$
insert	$O(n)$
delete	$O(n)$
space	$O(n)$

Taken from Skilled.dev

Operations

Pros 🔥

Flexible Size / Distributed

Data does not need to be sequential in memory.
We can arbitrarily add/remove items in a linked list as long as we have a way to point to the next item.

Ordered

Linked lists provide order by pointing to the `next` item in the linked list.

Fast Insert / Delete

If we have access to a node (typically the head or tail), we get fast inserts and deletes because the only thing that is required is updating a pointer.

Cons 🤔

Slow Lookups

There is no direct access to data (no keys/indexes).
We must iterate through the list sequentially to perform a lookup.

Taken from Skilled.dev

```
class DoublyNode {
  constructor(data) {
    this.data = data;
    this.next = null;
    this.prev = null;
  }
}

class DoublyLinkedList {
  constructor(data) {
    const initialNode = new DoublyNode(data);

    this.length = 0;
    this.head = initialNode;
    this.tail = initialNode;
  }

  // O(1)
  prepend(data) {
    const prependedNode = new DoublyNode(data);
    const previousHead = this.head;

    this.head = prependedNode;
    this.head.next = previousHead;

    previousHead.prev = this.head;

    this.length++;

    return this.head;
  }
}
```

Implementation

- Taken from Skilled.dev

```
// O(1)
append(data) {
  const appendedNode = new DoublyNode(data);
  const previousTail = this.tail;

  this.tail = appendedNode;
  this.tail.prev = previousTail;

  previousTail.next = this.tail;

  this.length++;

  return appendedNode;
}
```